

ShopFlow Curriculum — Part E: Career & Interview Readiness

Reviewer edition — annotated by an EM / Senior SWE / Technical Recruiter / FAANG interviewer panel, with an expanded final phase.

PANEL VERDICT

The original Phase 10 gets a motivated student to a competent first-pass resume and a passable interview. It under-prepares them for the parts of hiring that are not purely technical: sourcing opportunities, surviving take-homes, negotiating, and standing out against hundreds of similar bootcamp/self-taught portfolios. Treat the additions below as required, not optional.

1. Panel Evaluation of the Existing Curriculum

Ten areas the prompt asked us to review, scored against what actually gets a candidate through internship/junior-role screens today.

Resume	Impact-bullet rewrite, quantify honestly, outside review.	No guidance on ATS parsing, one-page discipline, or tailoring per posting.	Add ATS-safe formatting pass + a tailoring step per application (Sec. 3, 6).
Portfolio	Case-study structure: problem, approach, stack, what you'd improve.	Single-project portfolio (ShopFlow only). Recruiters expect range: a UI-heavy piece, a data/API piece, a scripting/CLI piece.	3-project portfolio minimum; add a second smaller project (Sec. 4).
GitHub	Pinned repos, READMEs, profile README, prune dead repos.	No commit-history hygiene, no contribution graph guidance, nothing on how reviewers actually skim a repo in 90 seconds.	Add a '90-second reviewer skim' checklist + commit-message standard (Sec. 6).
LinkedIn	Headline/About refresh, featured project, accurate stack.	Treated as a one-time pass, not an ongoing channel; nothing on recruiter outreach or connection strategy.	Add a Networking & Recruiter Communication track (Sec. 2).
System Design	Right scope for junior level: caching, load balancing, scaling narrative.	No API design (REST conventions, versioning) and no 'design a feature' style prompt, which is now common even at junior level.	Add a lightweight API-design project alongside the scaling one.
Security	Solid OWASP self-audit grounded in the student's own code.	No secure-code-review exercise on someone else's code, which is what many interviews actually test.	Fold into the new Code Review track (Sec. 2).
Behavioral	STAR bank with real ShopFlow stories, timed to ~90 seconds.	No coverage of 'why this company,' salary questions, or how to handle a question with no good story.	Add negotiation and company-research prep (Sec. 2, 5).
Technical Interview	DSA + system-design-lite mock, feedback on communication.	Only one mock format. No pair-programming, no take-home, no unfamiliar-codebase exercise — all common junior formats.	Add 4 additional interview formats (Sec. 5).
Mock Interviews	Uses a peer/mentor/AI tool; asks for feedback beyond correctness.	One mock, one time. No escalation, no rubric, no video review.	Structured 4-round mock circuit with rubric (Sec. 5).
LeetCode Prep	Realistic weekly cadence, spaced retention check.	No mapping from problems to actual company patterns, and no plan for when a student stalls on a pattern.	Add pattern-tagging + NeetCode 150-style checklist reference.

2. Additional Topics the Curriculum Is Missing

These close the gap between “can code” and “can get and keep an offer.” Each is scoped to 1–3 hours, not a new multi-week phase.

Topic	Why it matters	What to actually do
Salary Negotiation	Most junior candidates leave money on the table by accepting the first number.	Research a real range (levels.fyi, Glassdoor) for the target role/location; write and practice one counter-offer script out loud.
Networking	Referrals convert at a far higher rate than cold applications.	Message 5 alumni/engineers at target companies with a specific, non-generic ask; attend one meetup or virtual event.
Personal Branding	A recognizable, consistent presence makes a candidate memorable across touchpoints.	Align resume, GitHub, LinkedIn, and portfolio site around one clear one-line positioning statement.
Open Source Contributions	A merged PR on a real project is stronger signal than another solo tutorial repo.	Find 2 “good first issue” labeled repos in your stack; submit one real PR, even a small one.
Technical Writing	Writing ability signals seniority potential and clarity of thought to interviewers.	Write one deep-dive post explaining a hard ShopFlow decision (e.g. why Redis, why this schema).
Blog Creation	A blog is a durable, linkable artifact recruiters and hiring managers can skim.	Stand up a simple blog on the portfolio site; publish the technical-writing piece above as post #1.
Personal Website Polish	The Part E portfolio project builds a site; it rarely gets a second, critical pass.	Run a Lighthouse/accessibility audit; fix mobile layout, load time, and broken links.
Interview Scheduling Strategy	Where a slot sits in the interviewer’s day and week measurably affects outcomes.	Learn to request mid-morning slots when possible and to always confirm timezone in writing.
Recruiter Communication	Most candidates either go silent or over-explain when talking to recruiters.	Draft and practice 3 templates: initial outreach, timeline-nudge follow-up, and offer/negotiation reply.
Code Review Skills	Giving and receiving review feedback is a daily junior-dev task, rarely rehearsed beforehand.	Review a peer’s PR (or an open-source PR) and leave 3 substantive, kind, specific comments.
Pair Programming	Many onsites and take-homes are now collaborative, not solo whiteboard sessions.	Pair with a peer for 45 minutes on a small feature; switch driver/navigator roles halfway through.
Reading Unfamiliar Codebases	First weeks on any job are 80% reading code you didn’t write.	Clone a mid-size open-source repo; in 30 minutes, map its structure and explain one feature’s data flow.
Take-Home Assignments	Extremely common at the junior level and structurally different from live interviews.	Complete one realistic 3-4 hour take-home under a real clock; submit with the README a reviewer expects.
Internship-Specific Prep	Internship processes (OA platforms, cohort timing, return-offer criteria) differ from full-time hiring.	Research your target companies’ OA tooling (HackerRank/Codility) and their return-offer conversion rate.

3. A Better Mock Interview Program

Replace the single generic mock with a **4-round circuit**, run over the final two weeks, each scored against a fixed rubric instead of vague feedback.

1 – DSA solo	15 min, 1 medium LeetCode problem, think-aloud	Problem clarification, verbalized approach before coding, no rambling, no guessing
2 – System Design	30 min, Design a feature ShopFlow doesn't have	Requirements gathering, questions asked first, tradeoffs stated, no rambling
3 – Pair/collab	45 min, add a small feature to a partner's unfamiliar codebase	Asks for feedback, communicates intent before typing, no rambling
4 – Behavioral	10 min, take-home debrief	STAR bank, then defend as pair, specific metrics, no rambling, no rambling

Escalation rule: record round 1 and round 4; a student who can’t watch their own recording without wincing isn’t done practicing that round.

4. A Stronger Portfolio Strategy

One capstone (ShopFlow) reads as “followed a tutorial for a year” even when it isn’t. Ship **ShopFlow plus two small, fast, contrasting projects** so the portfolio demonstrates range, not just endurance.

Project	Purpose	Timebox
ShopFlow (existing)	Depth: full-stack ownership, auth, data modeling, deployment (scaling story).	1 year
A tiny public API + docs	Shows API design taste and technical writing outside a weekend app.	1 weekend
A CLI tool or automation script solving a showstopper problem	Shows script/tooling instincts and initiative unprompted by a course.	1 week
One merged open-source PR	External validation someone else's codebase and repository accepted your work.	Ongoing

5. The Ideal Final Phase: Phase 10 Redesigned

Same 3–4 week envelope, restructured from 4 topics to **6**, with the new material folded in rather than bolted on.

Phase	10 (redesigned)
Topics	6
Duration	4 weeks (unchanged envelope, denser content)
Projects	3–5 per topic, kept hands-on
New readiness bar	Explain ShopFlow's architecture out loud • defend its decisions • live URL • medium LeetCode under time • one merged open-source PR

Week 1	System Design & API Craft	Original scaling content + REST/API design conventions and a “design a new feature” exercise.
Week 1	Security & Code Review	Original OWASP self-audit + reviewing a partner's PR and one external open-source PR.
Week 2	Resume, Portfolio & Brand	Original resume/portfolio work + the two small contrasting projects, blog post #1, and a personal-brand pass across every profile.
Week 2	Networking & Recruiter Communication	New: outreach templates, referral requests, recruiter follow-up scripts, one real open-source PR submitted.
Week 3	Applied Interview Skills	New: one timed take-home, one pair-programming session, one unfamiliar-codebase reading exercise, interview-scheduling strategy.
Week 4	Interview Prep & Negotiation Sprint	Original LeetCode/STAR/mock content, run as the 4-round mock circuit, plus salary-negotiation research and script practice.

■ Study Resources (added)

- levels.fyi & Glassdoor — salary research — <https://www.levels.fyi>
- Up For Grabs — beginner-friendly open-source issues — <https://up-for-grabs.net>
- REST API design guidance — <https://restfulapi.net>
- GitHub Docs: About pull requests — <https://docs.github.com/en/pull-requests>

6. Part E — Redesigned Progress Tracker

■ System Design & API Craft	■ Interview Prep & Negotiation Sprint
■ Security & Code Review	■ 3-project portfolio live (ShopFlow + API + CLI/script)
■ Resume, Portfolio & Brand	■ One merged open-source PR
■ Networking & Recruiter Communication	■ 4/4 mock interview rounds passed
■ Applied Interview Skills	■ One completed timed take-home

Job-ready bar (updated): explain ShopFlow's architecture out loud, defend its technical decisions, point to a live deployed URL with a second and third portfolio piece behind it, solve a medium LeetCode problem under time pressure, show one merged open-source PR, and clear all four mock-interview rounds. That combination — not any single phase alone — is what gets you hired.